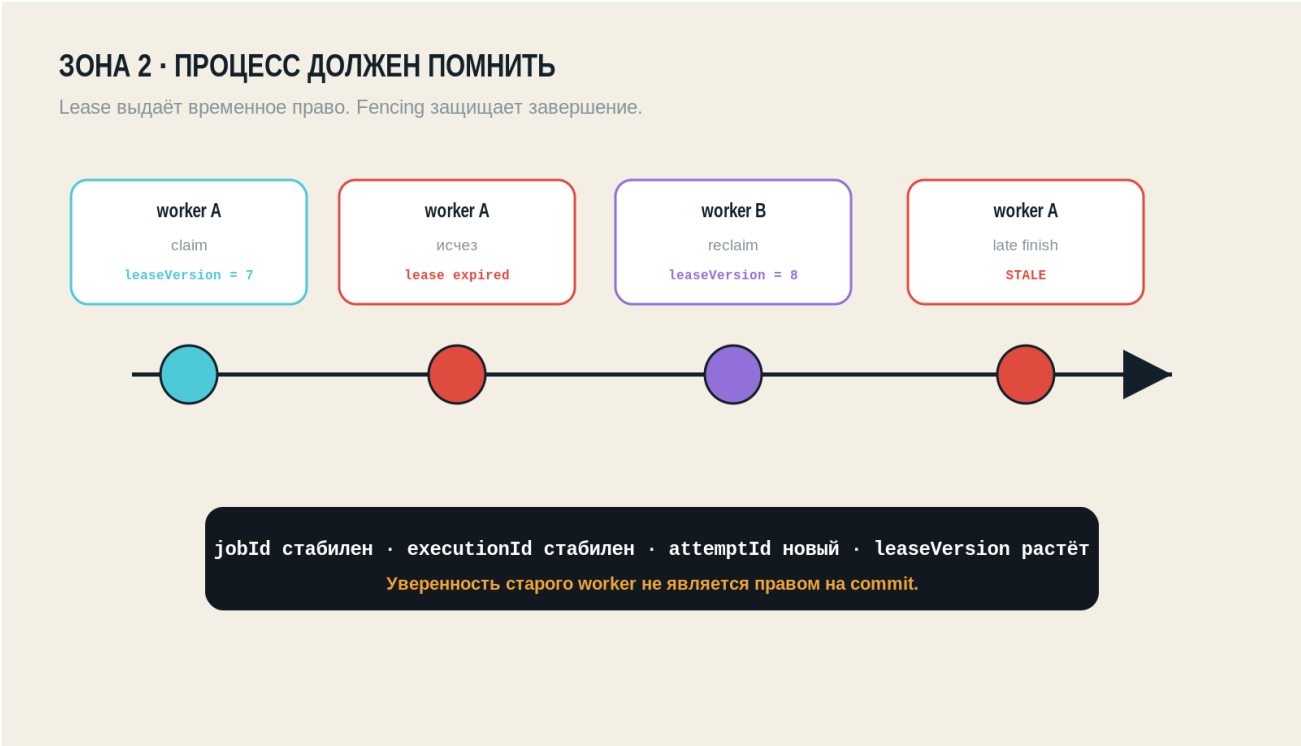


MODULEDEV · ESCAPE FROM LOCALHOST

Зона 2. Память процесса — практическое задание

Персистентное workflow-ядро



ЗОНА	Зона 2. Память процесса
ТЕХНИЧЕСКАЯ ТЕМА	Персистентное workflow-ядро
ФОРМАТ	лекция · самостоятельный проект · black-box приёмка

Цель

Построить общий C#-исполнитель версионированных процессов. Определения, состояние, jobs, attempts, signals и история хранятся в PostgreSQL, переживают recreate worker и не зависят от имён предметных карт или actions.

Срок и сдача

Работа выполняется с 28 августа по 3 сентября 2026 года. Дедлайн — 3 сентября, 23:59 МСК.

Проверяется продолжение того же репозитория и полный SHA commit. До дедлайна участник отправляет куратору URL, ветку и SHA и заранее предоставляет доступ. В сдачу не включаются .env, реальные секреты, build output, логи и сгенерированные отчёты проверки.

Проверяемый результат

После сборки проверяющий контур:

1. Публикует PostgreSQL action с неизвестными C#-коду именами и outcomes.
2. Публикует и активирует неизвестную workflow-карту.
3. Запускает process через CLI без изменения и пересборки API/worker.
4. Доводит automatic step до устойчивого ожидания и затем до end.
5. Останавливает worker после claim, дожидается reclaim и отклоняет stale completion.
6. Доказывает один предметный эффект, pinned flow version и append-only attempts после recreate worker.

Связь workflow-модели с BPMN

Workflow-модель курса задаёт четыре базовых понятия:

flow -> versioned process definition
step -> process node
route -> transition by outcome
task -> registered action invocation

BPMN используется как язык рассуждения о процессе:

BPMN-понятие	Подмножество course-1
Start event	start_step
Service task	automatic
Message catch event	wait_signal
User task	manual
End event	end
Exclusive routing	Переход по конечному outcome

Это не задание на BPMN-движок. Импорт BPMN XML, parallel gateways, timers, boundary events, compensation, subprocesses и произвольные expressions не требуются.

Основание

Используется action runtime недели 1. Automatic task не вызывает предметную функцию напрямую и не содержит C# handler. Он хранит:

- service = postgres;

- module, action, action_version;
- required policy;
- timeout и bounded retry policy;
- mapping process data в action payload;
- переходы по опубликованным outcomes.

Worker создаёт trusted context principal workflow-worker, добавляет processId, jobId, executionId, attemptId и deadline, затем использует тот же shared C# action executor и api.invoke.

Обязательный интерфейс запуска

Решение продолжает test adapter недели 1. В Compose обязательны сервисы:

Сервис	Обязательное свойство
gateway	Единственная внешняя точка на host-порту 8080
api	Внутренний C# action runtime без host-порта
cli	Принимает course CLI commands как container entryptpoint
postgres	PostgreSQL, база course, named volume
worker-a	C# Workflow.Worker, lease owner worker-a, без host-порта
worker-b	Тот же C# image, lease owner worker-b, без host-порта

Чистый запуск:

```
docker compose up -d --build
```

Открытая проверка:

```
./check.sh
```

Проверяющий контур не исполняет course.sh сдачи на host. Commands передаются entryptpoint сервиса cli; trusted fixtures монтируются read-only. После docker compose up не требуется ручной DML, публикация встроенных maps или настройка credentials.

Проверка отклоняет Compose-конструкции, которые выходят за границы отдельного project: external resources/providers/links, любые bind mounts и file-backed configs/secrets, внешние logging drivers, Docker socket/API, host namespaces, privileged/device/capability/custom-runtime options, include/extends, внешние build contexts, дополнительные build tags/exporters, нестандартные resource drivers, любые volume driver_opts и network driver_opts/ipam. Явные container_name, image и имена локальных resources допустимы: trusted override заменяет их изолированными именами на время проверки. Bootstrap, migrations, встроенные manifests и maps копируются в images при сборке; проверочные fixtures монтирует только checker после cold build.

Для машинной проверки C# Dockerfile API и worker должен запускать dotnet, DLL или self-contained binary с именем assembly проекта. Эффективный финальный stage, выбранный build.target или последним FROM, либо основан на mcr.microsoft.com/dotnet/*, либо получает запускаемый runtime artifact через COPY --from из такого build stage.

Обязательная модель

Definition

Сущность	Назначение
flow_definition	Стабильное имя процесса
flow_version	Неизменяемая опубликованная версия карты
step_definition	Тип шага и параметры
transition_definition	Переход для конкретного outcome
task_definition	Закреплённый action contract и execution policy

Публикация создаёт immutable version, но не активирует её. flow activate атомарно выбирает ровно одну published active version для новых instances. Уже запущенный process навсегда остаётся на своей версии.

Runtime state

Сущность	Назначение
process_instance	Экземпляр конкретной версии карты
step_instance	Фактическое состояние шага
workflow_job	Готовое, арендованное или отложенное задание
task_attempt	Одна попытка выполнения job
workflow_signal	Идемпотентно принятый локальный сигнал
workflow_event	Append-only история переходов

Process states: CREATED, RUNNING, WAITING_SIGNAL, WAITING_MANUAL, COMPLETED, FAILED.

Job states: READY, LEASED, RETRY_WAIT, SUCCEEDED, DEAD.

Типы шагов

Тип	Поведение недели 2
automatic	Создаёт job и вызывает зарегистрированный action
wait_signal	Устойчиво ждёт signal и продолжает process после его принятия
manual	Создаёт ожидающий step и устойчиво переводит process в WAITING_MANUAL

Тип	Поведение недели 2
end	Завершает технический process с объявленным outcome

На неделе 2 manual проверяется до состояния WAITING_MANUAL, включая persistence после recreate. HTTP-action ручного решения, principal/reason и конкурентные решения реализуются на неделе 3.

Формат и публикация карты

CLI-контракт:

```
./course.sh flow validate <file>

./course.sh flow publish <file>

./course.sh flow list

./course.sh flow activate <flow> --version <version>

./course.sh flow start <flow> --business-key <key> [--data <file>]

./course.sh flow get <process-id>

./course.sh flow signal <process-id> --type <type> --message-id <id> --payload <file>
```

CLI пишет в stdout ровно один JSON document, diagnostics — только в stderr. Ошибка даёт non-zero exit code и error envelope с meta.contractVersion = course-1.

Обязательные поля result:

Команда	Поля
flow publish	resource="flow", operation="published", flowName, flowVersion
flow activate	resource="flow", operation="activated", flowName, flowVersion
flow start	resource="process", operation="started", processId, flowName, flowVersion, state
flow get	resource="process", processId, flowName, flowVersion, state, currentStepKey
flow signal	resource="signal", processId, messageId, signalType, status со значением accepted или duplicate

List-команды возвращают result.items. processId сериализуется UUID-строкой. Action workflow.get принимает {"processId":"..."} с policy workflow:read и возвращает внутри result объект process и массивы steps, jobs, attempts.

flow validate не изменяет данные. Повторная идентичная публикация возвращает исходный result; попытка опубликовать другой contract под тем же flow_name/version возвращает conflict. flow start идемпотентен по паре flow_name/business_key: одинаковые data возвращают прежний process,

изменённые data дают conflict даже после смены active version. Без --data используется {};
отсутствие active version возвращает контролируемую ошибку. До возврата flow start process и первый waiting state или job уже зафиксированы.

Локальная trusted command flow signal пишет workflow_signal без integration Inbox недели 3.
message-id глобально уникален: duplicate возвращается только при совпадении process, type и body; любое расхождение даёт conflict без новой или изменённой signal-строки. Сигнал типа, объявленного в pinned map, можно принять до входа в соответствующий wait_signal: он хранится как ACCEPTED и атомарно применяется при входе в ожидание. Новый сигнал неизвестного pinned map типа отклоняется. Сигнал для уже завершённого process сохраняется, но process не двигает.

Semantic validation

До публикации validator проверяет:

- один существующий start_step;
- уникальные step keys;
- хотя бы один достижимый end;
- достижимость всех steps;
- отсутствие циклов и тупиковых non-end paths;
- существование и enabled state service/action/version;
- точное равенство множеств task.required_policy и action required_policy;
- достаточность server-side scopes workflow-worker;
- ровно один transition для каждого action outcome и каждого allowed manual outcome;
- отсутствие transitions из end;
- корректность mapping, timeout и retry policy;
- отсутствие неизвестных полей по JSON Schema course-1.

Достижимость manual проверяется по графу, а не по наличию команды завершения недели 3.

Mapping

input_mapping имеет направление target payload pointer -> source process data pointer. Оба значения являются JSON Pointer по RFC 6901. input_constants задаёт исходный JSON object payload.

Target pointers mappings не должны пересекаться между собой или с уже заданными constants. Пересечением считаются равенство и отношение предок/потомок по сегментам JSON Pointer: /a пересекается с /a/b, но не с /ab. Отсутствующий source во время исполнения даёт non-retryable workflow.mapping_missing; action не вызывается. Сформированный payload валидируется request schema action до api.invoke.

Retry

- `max_attempts` включает первое исполнение `action` и находится в диапазоне 1–10;
- `delays_ms` содержит ровно `max_attempts - 1` значений;
- после `retryable failure N` используется `delays_ms[N - 1]`;
- повторяются `error envelope` с `retryable=true` и `runtime timeout`;
- `mapping error`, `unknown outcome` и `response contract violation` не повторяются;
- после исчерпания попыток `job` становится `DEAD`, `step/process` — `FAILED`, `history` получает `TaskFailed`;
- `claim` с истёкшей `lease` остаётся `attempt` со статусом `STALE`, увеличивает `attempt_count`, но не расходует `failure budget max_attempts`; `reclaim` создаёт новую `attempt`;
- `jitter` и бесконечные `retries` не требуются.

Lease, attempts и fencing

Worker захватывает `ready jobs` короткой транзакцией без блокировки на время `action`. `FOR UPDATE SKIP LOCKED` является рекомендуемым, но не обязательным способом.

Идентификаторы:

Поле	Семантика
<code>jobId</code>	Одна логическая работа
<code>executionId</code>	Стабильный <code>idempotency key</code> предметного эффекта всех <code>retries job</code>
<code>attemptId</code>	Уникальная конкретная попытка
<code>leaseVersion</code>	Возрастающая версия права на завершение

При `reclaim` сохраняются `jobId` и `executionId`, создаётся новый `attemptId`, увеличивается `leaseVersion`. `Finish` принимается только при совпадении `jobId`, `owner`, `leaseVersion` и ожидаемого `state`. `Stale finish` не меняет `job`, `process` или предметные данные.

Роль `workflow_worker` имеет `EXECUTE` ровно на `workflow.claim_jobs`, `api.invoke`, `workflow.finish_job`, `workflow.fail_job` и не имеет прямого `DML` к предметным и `workflow`-таблицам. Для функций `schemas api` и `workflow` учитываются также права, унаследованные от `PUBLIC`, поэтому лишний `default EXECUTE` нужно отозвать. `workflow.claim_jobs` возвращает `worker` закреплённые `action version`, `schemas`, `outcomes` и `policy`, необходимые `shared executor`; отдельное чтение каталога роли не требуется. Имя роли является частью проверочного контракта.

Транзакционные границы

Успешная `automatic attempt`:

BEGIN

`api.invoke(... trusted context with executionId ...)`

`validate envelope, outcome and result schema`

`workflow.finish_job(jobId, owner, leaseVersion, outcome, result)`

COMMIT

Обязательно:

- action effect и successful finish_job находятся в одной Npgsql transaction;
- finish_job повторно проверяет owner, leaseVersion, state и outcome;
- продвижение process, завершение step, event и создание следующего job фиксируются атомарно;
- error envelope или contract violation откатывают action transaction;
- после rollback отдельный fail_job сохраняет attempt и retry schedule;
- неизвестный runtime outcome не выбирает fallback transition;
- история не обновляется и не удаляется, а только дополняется.

Test profile и failpoints

Для открытых и hidden аварийных сценариев обязательна поддержка:

`COURSE_TEST_PROFILE=1`

`COURSE_FAILPOINT=after_job_claim`

`COURSE_FAILPOINT=after_action_before_finish`

Failpoint	Точная граница
after_job_claim	После commit lease и attempt, до api.invoke
after_action_before_finish	После action effect и contract validation внутри transaction, до finish_job и commit

При достижении failpoint worker пишет одну structured log entry:

```
{"event": "failpoint.reached", "name": "after_job_claim", "instanceId": "worker-a"}
```

Затем worker блокируется до принудительной остановки. Checker ждёт acknowledgement, а не использует случайный sleep.

Checker включает test profile и передаёт одно значение COURSE_FAILPOINT обоим worker. Победителя определяет атомарный claim, а не отдельная переменная конкретного container.

Для fencing probe test adapter предоставляет command:

```
./course.sh flow test-finish <job-id> \  
  
--owner <owner> \  
  
--lease-version <version> \  
  
--outcome <outcome> \  
  
--result <file>
```

Command доступна только при COURSE_TEST_PROFILE=1, не публикуется по HTTP, вызывает ту же production finish boundary и не разрешает DML или произвольный SQL target. Stale request возвращает non-zero exit code и workflow.lease_state.

Test profile: lease 2 секунды, poll interval не более 100 ms, общий timeout одного hidden scenario 30 секунд. Интервалы задаются конфигурацией; production profile может быть консервативнее.

Диагностика и evidence

Обязательны compact flow get, action workflow.get, server-side IDs и safe error codes без stack trace, credentials и полного payload. Image digest worker до и после hidden publication должен совпасть.

Read-only schema autocheck публикует views. Указанные колонки и типы обязательны, но их порядок и дополнительные диагностические колонки не являются контрактом. Все relation имеют тип обычного view, не materialized view. timestampz означает timestamp with time zone; неуказанные идентификаторы имеют тип uuid.

View	Обязательные колонки
action_definitions	module text, action text, version integer, http_method text, target_schema text, target_function text, outcomes jsonb, enabled boolean, is_default boolean
action_dispatches	correlation_id, request_id text, module text, action text, version integer, principal text, payload_hash text, status text, outcome text, occurred_at timestampz
flow_versions	flow_name text, flow_version integer, status text, is_active boolean, published_at timestampz
processes	process_id, business_key text, flow_name text, flow_version integer, state text, current_step_key text, created_at timestampz, updated_at timestampz
steps	step_instance_id, process_id, step_key text, step_type text, state text, outcome text, entered_at timestampz, completed_at timestampz
jobs	job_id, process_id, step_instance_id, execution_id, state text, lease_owner text, lease_version bigint, lease_until timestampz, attempt_count integer, next_attempt_at timestampz
attempts	attempt_id, job_id, execution_id, lease_version bigint, attempt_number integer, status text, outcome text,

View	Обязательные колонки
	error_code text, started_at timestampz, finished_at timestampz
signals	message_id text, process_id, signal_type text, body_hash text, status text, received_at timestampz
workflow_events	event_id, process_id, step_instance_id, event_type text, occurred_at timestampz

Enum-подобные state/status/type используют uppercase ASCII: flow_versions.status=PUBLISHED; process states перечислены выше; steps.step_type — AUTOMATIC, WAIT_SIGNAL, MANUAL, END; step states — PENDING, READY, RUNNING, WAITING, COMPLETED, FAILED; job states перечислены выше; attempt statuses — RUNNING, SUCCEEDED, FAILED, STALE; signal statuses — ACCEPTED, APPLIED. workflow_events.event_type является именем доменного события в PascalCase; при исчерпании retry обязателен TaskFailed.

Workflow-вызов action сохраняет action_dispatches.request_id=executionId и principal=workflow-worker. Строка успешного dispatch фиксируется в общей transaction action/finish; rollback не оставляет dispatch. Из error codes недели 2 фиксированы workflow.mapping_missing и workflow.lease_stale; остальные используют safe lowercase dotted identifiers и не проверяются по конкретному имени, кроме семантики conflict.

Authoritative checker использует CLI и views, а не физические таблицы и имена C# classes.

Практическое задание «Worker можно заменить»

Реализуйте:

- migrations workflow-области и минимально привилегированные roles;
- schema и semantic validator workflow-map course-1;
- publish/list/activate/start/get/signal CLI;
- C# Workflow.Worker и два Compose instances одного image;
- четыре типа steps в границах недели 2;
- persistent process, step, job, attempt, signal и event state;
- lease, expiry, reclaim и fencing;
- bounded timeout/retry;
- shared action executor недели 1;
- action workflow.get;
- idempotent PostgreSQL test action;
- workflow-smoke versions 1 и 2;
- open tests через ./check.sh;

- C4 Container update и ADR о lease/fencing.

workflow-smoke v1 должна пройти automatic -> wait_signal -> end. Отдельная ветка или карта должна дойти до WAITING_MANUAL. Version 2 меняет последующее поведение так, чтобы старый instance остался на v1, а новый доказуемо использовал v2.

Открытая проверка

Участник заранее получает executable scenarios:

Область	Что проверяется
Admission	Compose seam, C# worker, PostgreSQL, clean start, repository hygiene
Map contract	Schema, graph, action/version, policy, outcomes, mapping и retry
Publication	Idempotent publish, conflict, atomic activate и pinned versions
Execution	Automatic, signal, end, manual wait и trusted context
Concurrency	Два worker, один effect, stable executionId
Recovery	Deterministic crash, reclaim, stale finish, rollback before finish
Evidence	Views, append-only attempts/events, safe errors и recreate

Hidden checker проверяет те же инварианты с новыми names, payload properties, outcomes, ordering и interleavings.

Критерии приёмки

- Hidden action и map выполняются без изменения или пересборки C#.
- Invalid maps отклоняются без side effects.
- Published version неизменяема; activate влияет только на новые instances.
- Automatic, wait signal, manual wait и end интерпретируются по данным map.
- Один business key не создаёт второй process.
- Два worker не создают два предметных эффекта одного job.
- Crash after claim приводит к reclaim после expiry.
- Stale completion отклоняется.
- Crash after action before finish оставляет ноль partial effects; retry создаёт один effect.
- Retry ограничен, а DEAD, FAILED и TaskFailed согласованы.

- READY, RETRY_WAIT, WAITING_SIGNAL и WAITING_MANUAL переживают recreate worker.
- Process, steps, jobs, attempts, signals и events согласованы и доступны через stable evidence seams.
- workflow_worker не выполняет прямой DML.

Артефакты недели

- работающий commit продолжения недели 1;
- migrations и роли workflow;
- validator/publisher и CLI;
- generic C# worker;
- maps v1/v2 и manual-wait scenario;
- открытые integration/concurrency/recovery tests;
- stable autocheck views;
- action workflow.get;
- C4 Container update;
- ADR о lease/fencing и at-least-once;
- README с секциями Архитектура, Запуск, Workflow-карты, Worker, Проверка, Диагностика, Ограничения;
- команды docker compose up -d --build и ./check.sh.

Не входит в неделю

- предметные payment maps;
- provider-simulator, Outbox, Inbox и HMAC receipt;
- завершение manual step через публичный action;
- BPMN XML import/export и графический designer;
- parallel/inclusive gateways, timers, cycles, subprocesses и compensation;
- migration запущенного process между versions;
- arbitrary expressions, code, URL или SQL из map;
- special C# branch по имени flow, step или action.

Если задание, contract reference и JSON Schema противоречат друг другу, сообщите куратору. Это дефект задания, а не повод угадывать hidden checker.